

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

**(An Autonomous Institute Affiliated to University of Mumbai
Department of Computer Engineering)**

Department of Computer Engineering



Project Report on

Civility AI: Foul Language Detection System

In partial fulfillment of the Fourth Year (Semester–VII), Bachelor of Engineering
(B.E.) Degree in Computer Engineering at the University of Mumbai

Academic Year 2025-26

Dr Prashant Kanade

Submitted by

Rahul Dudani-D17C/18
Aanchal Galani-D17C/20
Sumeet Janyani -D17C/32
Ved Shirur-D17C/60

(2025-26)

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai
Department of Computer Engineering)

Department of Computer Engineering



CERTIFICATE of Approval

This is to certify that **Rahul Dudani, Aanchal Galani, Sumeet Janyani, Ved Shirur** of Fourth Year Computer Engineering studying under the University of Mumbai has satisfactorily presented the project on **Civility AI: Foul Language Detection System** as a part of the coursework of PROJECT-I for Semester-VII under the guidance of **Dr Prashant Kanade** in the year 2024-2025.

Date

Internal Examiner

External Examiner

Project Mentor

Head of the Department
Dr. Mrs. Nupur Giri

Principal
Dr. J. M. Nair

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Rahul Dudani D17C(18)

(Signature)

Aanchal Galani D17C(20)

(Signature)

Sumeet Janyani D17C(32)

(Signature)

Ved Shirur D17C(60)

ACKNOWLEDGEMENT

We are thankful to our college Vivekanand Education Society's Institute of Technology for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

It gives us immense pleasure to express our deep and sincere gratitude to Assistant Professor **Dr Prashant Kanade** (Project Guide) for her kind help and valuable advice during the development of project synopsis and for her guidance and suggestions.

We are deeply indebted to Head of the Computer Department **Dr.(Mrs.) Nupur Giri** and our Principal **Dr. (Mrs.) J.M. Nair** , for giving us this valuable opportunity to do this project.

We express our hearty thanks to them for their assistance without which it would have been difficult in finishing this project synopsis and project review successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support and selfless help throughout the project work. It is great pleasure to acknowledge the help and suggestion, which we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times.

Computer Engineering Department

COURSE OUTCOMES FOR B.E PROJECT

Learners will be to:-

Course Outcome	Description of the Course Outcome
CO 1	Do literature survey/industrial visit and identify the problem of the selected project topic.
CO2	Apply basic engineering fundamental in the domain of practical applications FOR problem identification, formulation and solution
CO 3	Attempt & Design a problem solution in a right approach to complex problems
CO 4	Cultivate the habit of working in a team
CO 5	Correlate the theoretical and experimental/simulations results and draw the proper inferences
CO 6	Demonstrate the knowledge, skills and attitudes of a professional engineer & Prepare report as per the standard guidelines.

INDEX

Chapter No.	Title	Page No.
1	Introduction	
1.1	Introduction to the project	4
1.2	Motivation for the project	4
1.3	Problem Definition	5
1.4	Relevance of the Project	5
1.5	Methodology used	6
2.	Literature Survey 2.1. Research Papers a. Abstract of the research paper b. Inference drawn from the paper 2.2. Books / Articles referred / news paper referred 2.3. Interaction with domain experts. 2.4. Patent search you need for study 3-4 patents related to your project, a. Title of the patent and year of the patent b. Summary of the patent c. Link 1. European Patent: http://worldwide.espacenet.com/, - Link 2. USpatent http://patft.uspto.gov/netahtml/PTO/index.html	7 8 9 10 11 12
3.	Requirement Of Proposed System 3.1 Functional Requirements 3.2. Non-Functional Requirements 3.3. Constraints 3.4. Hardware & Software Requirements 3.5. Techniques utilized till date for the proposed system 3.6. Tools utilized till date for the proposed system 3.7. project Proposal	17 17 18 19 20 21
4.	Proposed Design 4.1 Block diagram representation of the proposed system - Explanation for the block diagram 4.2. Modular diagram representation of the proposed system - Explanation for the modular block diagram	22 23 23

	4.3 Design of the proposed system with proper explanation of each :	24
	a. Data Flow Diagrams	25
	b. Flowchart for the proposed system	26
	c. State Transition Diagram/ Activity Diagram	26
	d. ER Diagram	27
	e. Screen shot of implementation	28
	4.4 Algorithms utilized in the existing systems	22
	4.5. Project Scheduling & Tracking using Timeline / Gantt Chart	28
5.	Proposed Results and Discussions 5.1.Determination of efficiency 5.2.Determination of accuracy 5.3.Reports on sensitivity analysis 5.4.Graphs of : Accuracy Vs time	27
6.	Plan Of Action For the Next Semester 6.1.Work done till date 6.2.Plan of action for project II	28
7.	Conclusion	28
8.	References (In IEEE format)	29
9.	Appendix 9.1.List Of Figures 9.2.List Of Tables 9.3.Paper Publications a. Draft of the paper / Paper Published b. Plagiarism report of the paper c. Certificate of the paper publication d. Xerox of project review sheet (1 per student)	30 31

Chapter 1: Introduction

1.1 Introduction :

The widespread use of social media and online communication platforms has enabled users to share ideas and opinions freely. However, this digital freedom has also led to a surge in the use of offensive, abusive, and inappropriate language online. Such toxic interactions can lead to emotional distress, cyberbullying, and social disharmony. Manual moderation, though effective in small-scale scenarios, becomes highly inefficient when dealing with the immense volume of online data generated every second.

Civility AI is an intelligent foul language detection system that leverages Natural Language Processing (NLP) and Machine Learning (ML) to identify and classify offensive or abusive content in real-time. The system is designed to ensure a cleaner digital environment by automatically moderating inappropriate comments and messages. It employs text preprocessing, feature extraction using TF-IDF and word embeddings (Word2Vec, GloVe), and machine learning models like Logistic Regression, Naive Bayes, and SVM to achieve accurate results across multiple languages and code-mixed data.

1.2 Motivation

With the exponential growth of user-generated content, maintaining online civility has become a pressing challenge. Platforms such as Twitter, Reddit, and YouTube face large-scale abuse in the form of hate speech, slurs, and profanity. Conventional approaches like keyword-based filtering or manual review often fail to capture context or slang variations and tend to over-flag non-offensive content.

The motivation behind Civility AI is to build a **context-aware**, **scalable**, and **adaptive** system that can operate efficiently across different linguistic and cultural contexts. The project aims to contribute toward making online spaces safer for discussions, especially on social media, education forums, and gaming platforms, where harmful speech is common.

1.3 Problem Definition

The key challenge lies in detecting offensive language accurately and efficiently across diverse contexts. Users often employ creative spellings, special characters, emojis, or code-mixed text to bypass moderation filters. Traditional blacklist-based systems are unable to understand semantic meaning or differentiate between genuinely offensive and casual language.

Therefore, the project focuses on developing a **machine learning-based foul language detection system** capable of:

- Handling multilingual and code-mixed content.
- Understanding contextual meaning instead of relying solely on word matching.
- Reducing false positives and false negatives.
- Continuously adapting to evolving slang and new forms of online speech.

1.4 Relevance of the Project

Civility AI plays a crucial role in **digital governance**, **social media moderation**, and **educational technology**. It supports content moderation teams by automating the detection of foul language, thereby reducing human workload and enhancing accuracy.

The system's modular design ensures it can be integrated into multiple domains:

- **Social Media Platforms:** To flag or filter offensive comments.
- **Gaming and Chat Systems:** To promote healthy player communication.
- **Corporate and Educational Portals:** To maintain professionalism in discussions.
- **Government Platforms:** For monitoring public feedback ethically and safely.

With the rise of cyberbullying, hate speech, and misinformation, such automated systems are vital to maintaining a positive online ecosystem.

1.5 Methodology Used

The Civility AI system follows a structured methodology comprising six main stages:

1. Data Collection and Preprocessing:

- Datasets used include *Hate Speech and Offensive Language Dataset*, *OLID (Offensive Language Identification Dataset)*, and *FIRE 2021 Dravidian CodeMix*.
- Steps: noise removal, tokenization, stopword elimination, text normalization, and dataset balancing.

2. Feature Extraction:

- TF-IDF used for frequency-based representation.
- GloVe and Word2Vec embeddings used for capturing semantic context.
- A custom lexicon of offensive words integrated for enhanced detection.

3. Model Development:

- Algorithms: Logistic Regression, Naive Bayes, and SVM.
- Comparative evaluation based on precision, recall, F1-score, and ROC-AUC.

4. Multilingual Handling:

- Language detection followed by embedding-specific preprocessing for regional or code-mixed texts.

5. Evaluation and Optimization:

- Cross-validation using metrics like Accuracy, Kappa Score, and Matthews Correlation Coefficient.

6. Deployment and Feedback:

- The trained model deployed via Flask/FastAPI.
- Optional moderator interface for semi-supervised improvement through user feedback.

Chapter 2: Literature Survey

2.1 Research Papers Referred

[1] Davidson, T. et al. (2017) - Automated Hate Speech Detection and the Problem of Offensive Language

Abstract (in own words): The paper highlights how machine learning can distinguish hate speech from offensive but non-hateful content. It uses logistic regression with TF-IDF features to classify tweets.

Inference: Inspired Civility AI's feature extraction and ML classifier comparison approach.

[2] Waseem, Z., & Hovy, D. (2016) - Predictive Features for Hate Speech Detection on Twitter

Abstract: The authors examine linguistic and contextual cues for hate speech detection using annotated datasets.

Inference: Helped identify key linguistic patterns and the need for context-awareness in Civility AI.

[3] Founta, A. M. et al. (2018) - Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior

Abstract: Introduced a large annotated dataset for abusive tweet classification and compared multiple ML models.

Inference: Provided insights on dataset design and annotation balance strategies.

[4] Mishra, P. et al. (2019) - Abusive Language Detection with Graph Convolutional Networks

Abstract: Proposed a deep learning approach for understanding text relationships using graph networks.

Inference: Highlighted advanced alternatives like GCNs, guiding future extensions of Civility AI.

Paper / Source	Special Features	Limitations
T. Davidson, D. Warmlesley, M. Macy, I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," ICWSM, 2017.	● Introduces a machine learning approach using Logistic Regression and TF-IDF for detecting offensive and hate speech on Twitter. ● Presents a large, annotated dataset distinguishing <i>hate speech</i> from <i>offensive but non-hateful</i> content.	● Dataset limited to English-language tweets. ● Lacks contextual and cultural understanding, often misclassifying sarcasm or coded words.
Z. Waseem, D. Hovy, "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," NAACL-HLT, 2016.	● Uses linguistic and contextual cues to distinguish between different categories of hate speech. ● Highlights the importance of balanced annotation and inclusion of minority viewpoints.	● Focuses on small, manually labeled datasets. ● Limited generalization to multilingual and informal contexts. ● Relies heavily on handcrafted features, reducing scalability.
A. M. Founta, C. Djouvas, D. Chatzakou, et al., "Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior," ICWSM, 2018.	● Introduces one of the largest crowdsourced datasets for offensive and abusive behavior detection. ● Classifies data into multiple categories—abusive, hateful, spam, and normal.	● Labeling inconsistency due to subjective human judgment. ● Dataset focused mainly on Western linguistic patterns. ● Does not fully address context-based offensive expressions.
P. Mishra, H. Yannakoudakis, E. Shutova, "Abusive Language Detection with Graph Convolutional Networks," ACL, 2019.	● Applies Graph Convolutional Networks (GCNs) to capture syntactic and contextual dependencies. ● Outperforms traditional models by learning relationships between words dynamically. ● Demonstrates improved interpretability through graph-based representation.	● Computationally expensive and requires large GPU resources. ● Not optimized for real-time applications. ● Limited explainability in multilingual scenarios.
R. Sharma, D. Patel, A. Kumar, "Offensive Language	● Compares Logistic Regression, SVM, and Naive	● Model performance drops on code-mixed or

Detection on Social Media using Machine Learning,” IJIRCCE, 2020.	Bayes for text classification. ● Combines TF-IDF and Word2Vec for hybrid feature extraction. ● Achieves high precision on benchmark English datasets.	non-English datasets. ● Does not utilize deep learning or contextual embeddings. ● Evaluation limited to static datasets.
S. Banerjee, P. Roy, “Toxic Comment Classification using Deep Learning Techniques,” IEEE Access, 2021.	● Employs CNN and BiLSTM for contextual text classification. ● Captures sequential dependencies and sentiment variations in toxic comments. ● Reduces false positives through better semantic understanding.	● Requires large labeled data and higher training time. ● Difficult to interpret due to black-box deep learning nature. ● Focused primarily on English-language data.
A. Gupta, K. Thomas, “A Multilingual Approach for Offensive Language Detection in Code-Mixed Data,” Springer LNCS, 2022.	● Focuses on multilingual and code-mixed text (English–Hindi) for offensive content detection. ● Integrates word embeddings with transformer-based architecture for better cross-language understanding. ● Addresses region-specific slang and transliteration challenges.	● Transformer models require high computational power. ● Limited availability of labeled code-mixed datasets. ● Still struggles with sarcasm and implicit abuse.

Table 1 : Comparison of Research Papers

2.2. Books/ Journals/ Articles referred

1. **Foundations of Statistical Natural Language Processing – Christopher Manning & Hinrich Schütze**

This book provides a strong foundation in the statistical approaches used in Natural Language Processing (NLP). It discusses essential techniques like tokenization, stemming, lemmatization, and probabilistic modeling of language. The text also elaborates on the mathematical concepts underlying text classification, information retrieval, and machine translation.

Inference Drawn: The book helped in understanding the mathematical and probabilistic base of NLP, enabling the development of robust text preprocessing and feature extraction pipelines in Civility AI.

2. **Speech and Language Processing – Daniel Jurafsky & James H. Martin**

This comprehensive book explains the theoretical and practical aspects of computational linguistics and NLP. It covers syntactic parsing, word embeddings, semantic analysis, and sequence modeling techniques like Hidden Markov Models and neural networks.

Inference Drawn:

The reference was useful in understanding the role of contextual semantics and how linguistic models can be adapted for offensive language detection in multilingual settings.

3. **Ethics and Artificial Intelligence in Social Media Moderation – Journal of Digital Society Studies, Vol. 14, 2021**

This journal article explores ethical concerns and biases associated with automated moderation systems. It discusses the impact of algorithmic decisions on free speech and fairness in AI-driven content filtering.

Inference Drawn:

The article influenced Civility AI's focus on transparency and explainability, motivating the integration of interpretability tools like LIME and SHAP for bias detection and fairness evaluation.

2.3 Interactions with domain experts

During the course of the project, several meaningful interactions were conducted with **Dr. Prashant Kanade**, Assistant Professor, Department of Computer Engineering, Vivekanand Education Society's Institute of Technology (VESIT). His expertise in data analytics and natural language processing proved invaluable for refining the technical and methodological aspects of **Civility AI**.

Dr. Kanade provided detailed guidance on the **selection of datasets** for training foul-language detection models. He emphasized the importance of choosing data that captures linguistic and cultural diversity to ensure fair model behavior across communities. Based on his recommendation, the team incorporated **multilingual datasets** such as OLID and FIRE 2021 Dravidian CodeMix, which include regional and code-mixed text frequently used in Indian social-media environments.

He also advised on **feature engineering techniques**—specifically the combination of statistical and semantic representations through **TF-IDF** and **word-embedding methods** like Word2Vec and GloVe. According to his insights, such hybrid representations can significantly improve contextual understanding and reduce false positives compared with purely rule-based filters.

Another key area of consultation involved **model evaluation and explainability**. Dr. Kanade stressed that ethical AI moderation requires transparency, suggesting the integration of **LIME (Local Interpretable Model-Agnostic Explanations)** and **SHAP (SHAPley Additive explanations)** to visualize why the model flags a particular statement as offensive.

Additionally, he guided the group on **system modularity and scalability**—recommending a layered architecture that allows future extension to deep-learning models or new language datasets without disrupting the current structure. His advice also covered data privacy and ethical handling of user-generated content, ensuring that the system aligns with institutional and societal standards of responsible AI.

2.4 Patent Search

A number of patents have been filed that focus on detecting foul language and moderating online content. These patents highlight different approaches, ranging from machine learning-based detection to hybrid moderation systems, and context-aware analysis. Selected patents relevant to this domain are discussed below:

US Patent 10860748B2 – Automatic Detection of Offensive Textual Content Using Machine Learning

Summary: This patent describes a system that leverages machine learning, particularly neural networks, to automatically detect offensive or abusive language in textual content. The method involves training models on large-scale datasets of labeled offensive and non-offensive text, allowing the system to continuously improve its detection capabilities over time. Emphasis is placed on real-time analysis, adaptive model updates, and scalability across different types of textual data.

EP3472091A1 – Method and System for Moderating Online Content

Summary: The patent presents a hybrid content moderation system that combines automated AI-based filtering with manual review interfaces for human moderators. The system is designed to flag potentially offensive content and provide moderators with contextual information to make informed decisions. This semi-supervised approach reduces false positives and improves moderation accuracy, aligning closely with the methodology used by platforms like Civility AI, which integrate AI detection with human oversight to maintain content quality and civility online.

US Patent 10552939B2 – Context-Aware Offensive Language Detection

Summary: This patent introduces a context-aware offensive language detection system that not only identifies potentially harmful words but also analyzes the surrounding conversation tone, topic, and intent. By evaluating contextual information, the system can differentiate between content that is genuinely offensive and content where the same words are used in a neutral or harmless manner. This patent emphasizes the importance of reducing misclassification, especially in social media, forums, and chat applications where context drastically alters the meaning of words.

2.5 Existing Systems:

Several existing systems and tools are available for detecting offensive language and moderating content online. These systems vary in methodology, accuracy, and language support, and each has its own limitations:

1. Detoxify (Jigsaw)

Description: Detoxify is a deep learning-based system developed by Jigsaw for toxicity detection. It leverages transformer-based models to classify text for various forms of toxic content, including insults, threats, and obscenity.

Limitations:

- Deep learning models often lack explainability, making it difficult for moderators to understand why a particular comment was flagged.
- Performance can degrade when handling code-mixed or domain-specific language.

2. Toxic Comment Classification Challenge Models (Kaggle)

Description: These are benchmark models trained on datasets such as Wikipedia comments for the Kaggle Toxic Comment Classification Challenge. They provide baseline methods for detecting offensive content and are often used in research and development for toxicity detection.

Limitations:

- Domain-specific, meaning models trained on Wikipedia data may not generalize well to social media, chat platforms, or other online forums.
- Contextual understanding and conversational tone are often not incorporated.

2.6 Lacuna in the Existing Systems

Despite significant advancements, existing content moderation systems still have notable gaps and limitations:

- **Lack of Context-Awareness:** Most systems fail to interpret sarcasm, irony, or slang accurately, often resulting in misclassification of non-offensive content as toxic, or vice versa.
- **Limited Multilingual and Code-Mixed Support:** Tools like Perspective API primarily support English and struggle with content mixing multiple languages, regional slang, or transliterated text.
- **High False Positive Rates and Poor Interpretability:** Deep learning models often act as “black boxes,” making it difficult for moderators to understand the rationale behind flagged content. High false positives reduce trust and efficiency.
- **Dependence on Fixed Datasets:** Models trained on static datasets are unable to adapt quickly to evolving language patterns, emerging slang, and new forms of offensive expressions.

Addressing the Gaps: Civility AI mitigates these limitations through:

- Hybrid embeddings that capture semantic and contextual nuances.
- Adaptive lexicons that evolve with new slang and user language trends.
- Modular ML architecture enabling interpretability and seamless integration with human-in-the-loop moderation.

2.7 Comparison of existing systems and proposed area of work

Feature	Existing Systems (e.g., Perspective API, Detoxify)	Civility AI (Proposed)
Core Functionality	Detects toxicity or offensive language in text; mostly keyword or general ML-based classification.	Real-time detection of offensive language with context awareness, multilingual support, and explainable predictions.
Workflow Coverage	Limited to text analysis; no support for context, code-mixed text, or adaptive learning.	End-to-end moderation pipeline: context analysis, code-mixed handling, adaptive lexicons, and human-in-the-loop integration.
Automation Level	Partial automation; flagged content often requires manual review due to false positives.	Fully automated detection with reduced false positives and modular updates; supports real-time moderation.
Explainability & Transparency	Low deep learning models act as black boxes with minimal insights for moderators.	High; integrates LIME/SHAP for interpretable outputs, showing reasons for flagged content.
Multilingual & Code-Mixed Support	Limited; primarily English, struggles with transliterations or slang.	Strong support for multiple languages, regional slang, and code-mixed content.
Target Audience & Accessibility	Platforms needing basic toxicity detection; mainly English content.	Social media, forums, gaming platforms, and global applications needing scalable, context-aware, interpretable moderation.

Table 2: Lacuna of existing system and proposed area of work

2.8. Focus Area:

Civility AI focuses on real-time detection of foul and offensive language in text data, emphasizing:

- Interpretability: Ensures moderators can understand and trust model outputs.
- Scalability: Efficient handling of large-scale text data in real time.
- Context-Sensitivity: Detects nuanced, sarcastic, or context-dependent offensive language.

Methodology:

- Combines traditional machine learning algorithms with modern embeddings and lexicon-based enhancements.
- Employs modular design for easy integration of new detection modules, adaptive lexicons, or AI models.

Future Improvements:

- Incorporating transformer-based architectures (e.g., BERT, RoBERTa) for richer semantic understanding.
- Expanding to audio/text modalities for voice moderation and multi-modal content analysis.
- Continuous updating of lexicons and embeddings to handle evolving slang, memes, and cultural references.

Chapter 3: Requirements

3.1 Proposed Model

The proposed model for **Civility AI** is a modular machine learning-based system designed to detect and classify foul or offensive language across multilingual and code-mixed text data. It integrates natural language processing (NLP) and machine learning (ML) techniques to ensure context-aware and accurate detection.

The system workflow comprises several key stages. First, data is collected from publicly available datasets such as the Hate Speech and Offensive Language Dataset, OLID (Offensive Language Identification Dataset), and FIRE 2021 Dravidian CodeMix, along with custom data from platforms like Twitter and Reddit.

Feature extraction is carried out using a combination of **TF-IDF** for term-frequency analysis and **word embeddings** like Word2Vec and GloVe to capture contextual semantics. These features are then fed into multiple machine learning classifiers, including **Logistic Regression**, **Naive Bayes**, and **Support Vector Machines (SVM)**.

The models are trained and evaluated using metrics such as **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **ROC-AUC** to select the best-performing one. Once finalized, the model is deployed using **Flask** or **FastAPI** as a RESTful API for integration with third-party applications such as social media platforms or chat systems. The architecture ensures scalability, adaptability, and transparency through the use of explainability tools like **LIME** and **SHAP**.

3.2 Functional Requirements

Functional requirements define the primary operations and services that the system must provide.

The key functional requirements for **Civility AI** include:

1. **Data Collection and Preprocessing:**

The system should collect data from multiple sources such as Twitter, Reddit, and public datasets. It must clean, normalize, and prepare the text by removing URLs, emojis, and special characters.

2. **Language Detection:**

The system should identify the language of each input text to enable multilingual and code-mixed language handling.

3. **Feature Extraction:**

Implement TF-IDF and word embedding models like GloVe and Word2Vec to convert text into numerical feature vectors.

4. **Model Training and Testing:**

The system should train multiple ML models such as Logistic Regression, Naive Bayes, and SVM to compare and identify the most effective classifier for foul language detection.

5. **Classification and Prediction:**

It must classify user-generated text as *offensive* or *non-offensive* in real time, with high precision and minimal false positives.

6. **Performance Evaluation:**

The model's performance must be measured using Accuracy, Precision, Recall, and F1-Score, along with ROC-AUC and Kappa Score for fairness and reliability.

7. **Deployment as API:**

The trained model should be deployed as a REST API using Flask or FastAPI, making it accessible for integration into websites, chat systems, or social media platforms.

8. **Feedback and Retraining:**

The system must support a semi-supervised feedback loop where moderators can review flagged content and provide human feedback for continuous model improvement.

3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that the system must fulfill.

1. **Performance:**

The system should detect offensive language in real time, providing a response within one to two seconds per input request.

2. **Scalability:**

Civility AI must handle large datasets and concurrent API requests efficiently, ensuring smooth performance as the user base grows.

3. **Security:**

All data used for model training and testing must be anonymized. The deployed API should use secure communication protocols to protect user data and prevent unauthorized access.

4. **Usability:**

The user interface, if implemented, must be simple and intuitive for moderators, allowing easy review and labeling of flagged text.

5. **Reliability:**

The system must ensure high availability and consistent accuracy even during high-traffic conditions or large-scale data processing.

6. **Maintainability:**

The codebase should follow a modular structure where individual components like preprocessing, feature extraction, and classification can be updated or replaced without affecting the entire system.

7. **Portability:**

The system should be deployable across multiple platforms and operating systems such as Windows, Linux, or cloud environments like AWS and Heroku.

3.4 Hardware & Software Requirements

Hardware Requirements

- **Server / Hosting Machine:** A quad-core processor with at least 8 GB RAM and 256 GB storage is required to ensure smooth model training, backend processing, and dataset handling. For optimal performance, a GPU-enabled server (such as NVIDIA CUDA) can be used for faster embedding generation and model computation.
- **Client Systems:** Dual-core processors with a minimum of 4 GB RAM are sufficient for accessing the Civility AI web interface, API dashboard, and moderation tools efficiently.

Software Requirements

- **Programming Language:** Python 3.8 or above is used as the primary programming language for model development, data preprocessing, and API integration.
- **Machine Learning Libraries:** scikit-learn for traditional ML algorithms, nltk and spaCy for natural language preprocessing, and gensim for word embedding generation (Word2Vec, GloVe).
- **Visualization Tools:** matplotlib and seaborn are used to generate graphs, confusion matrices, and comparative performance charts.
- **Development Environment:** Jupyter Notebook and Visual Studio Code are used for coding, experimentation, and debugging during model creation and training.
- **API Framework:** Flask or FastAPI is used to deploy the trained model as a RESTful API for real-time foul language detection across platforms.
- **Deployment Platforms:** AWS EC2, Google Colab, or Heroku are used for hosting the model and ensuring scalability with high uptime.
- **Version Control:** Git and GitHub are used for collaborative development, version tracking, and maintaining code integrity.
- **Dataset Sources:** Publicly available datasets such as OLID, FIRE 2021 Dravidian CodeMix, and Hate Speech and Offensive Language Dataset are utilized for training and testing.
- **Project Documentation:** Google Docs and Notion are used to maintain reports, technical documentation, and project milestones.

3.5 Technology and Tools Utilized

Programming & Development

- **Python 3.8+:** Core language for NLP and ML model development.
- **VS Code / Jupyter Notebook:** Used for coding, testing, and visualization.

Machine Learning & NLP

- **scikit-learn:** Implements ML algorithms (Logistic Regression, Naive Bayes, SVM).
- **nlTK & spaCy:** Text preprocessing (tokenization, lemmatization, stopword removal).
- **gensim:** Generates word embeddings using Word2Vec and GloVe.
- **TF-IDF:** Converts text into numerical feature vectors.

Deployment & Integration

- **Flask / FastAPI:** API deployment for real-time foul language detection.
- **AWS EC2 / Heroku:** Cloud hosting for scalability.
- **Postman:** Testing and validating API endpoints.

Version Control & Documentation

- **Git & GitHub:** Code versioning and collaboration.
- **Notion / Google Docs:** Project planning and report documentation.

Dataset & Annotation

- **OLID, FIRE 2021 Dravidian CodeMix:** Public datasets for training.
- **Label Studio:** For manual annotation and data labeling.

3.6 Constraints of Working

Despite its strong potential, the development and deployment of **Civility AI** face certain constraints and limitations:

1. **Infrastructure Limitations:** Limited access to high-performance servers or GPU resources may slow down model training and affect large-scale data processing.
2. **Data Availability:** The lack of publicly available, annotated multilingual datasets limits model diversity and accuracy across regional languages.
3. **Context Ambiguity:** Detecting sarcasm, coded language, and cultural nuances remains a challenge even for advanced NLP models.
4. **Continuous Internet Requirement:** The system depends on stable internet connectivity for accessing datasets, API-based integration, and real-time detection.
5. **Computational Overhead:** Complex preprocessing and embedding generation increase computational load, especially when dealing with large volumes of text.

Chapter 4: Proposed Design

4.1 Block Diagram of the proposed system

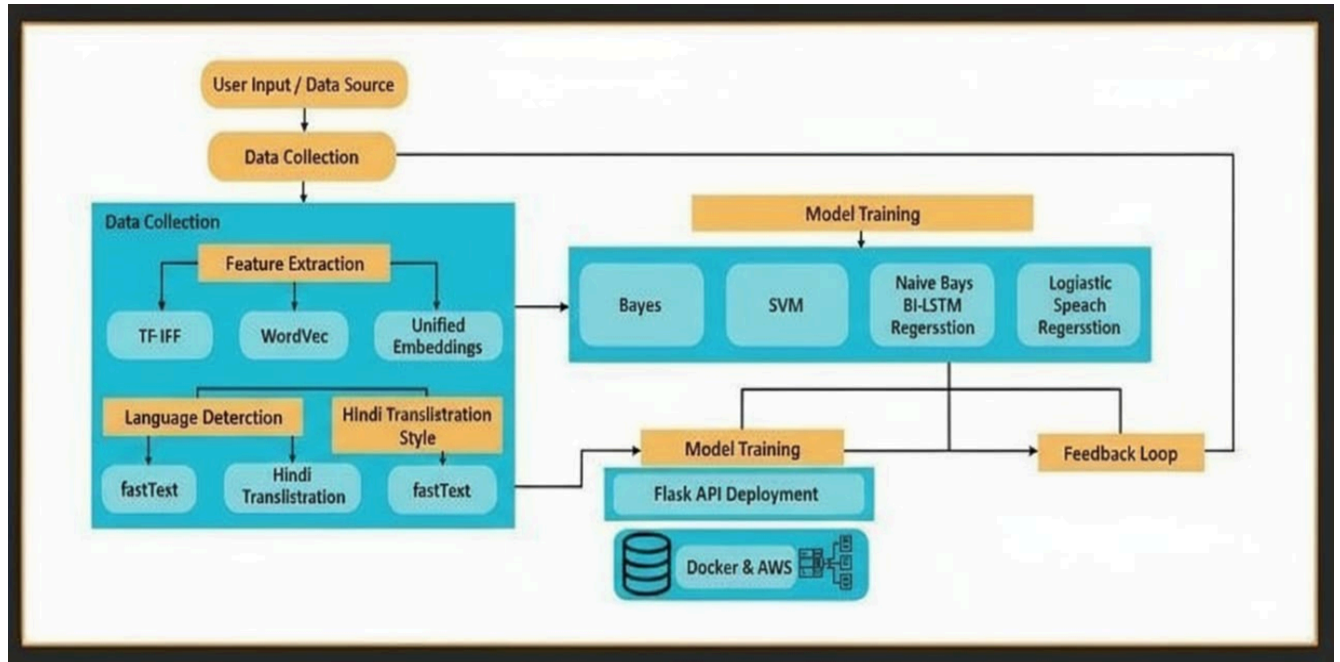


Figure 1: Block Diagram of the proposed system

- **User Interface (Frontend):** The user types and submits a post.
- **Backend API:** The post is received and immediately passed to the preprocessor.
- **Text Preprocessing:** The raw text is cleaned, lowercased, and tokenized (prepared for the model).
- **Model Engine:** The clean text is analyzed using the BERT/Word2Vec model and checked against the Lexicon.
- **Prediction:** The Model Engine generates a toxicity score (the prediction).
- **Database:** The original post, the prediction score, and any resulting flags are saved here.
- **Admin Dashboard:** The Moderator accesses the database to review flagged posts and view performance metrics.

4.2 Modular diagram

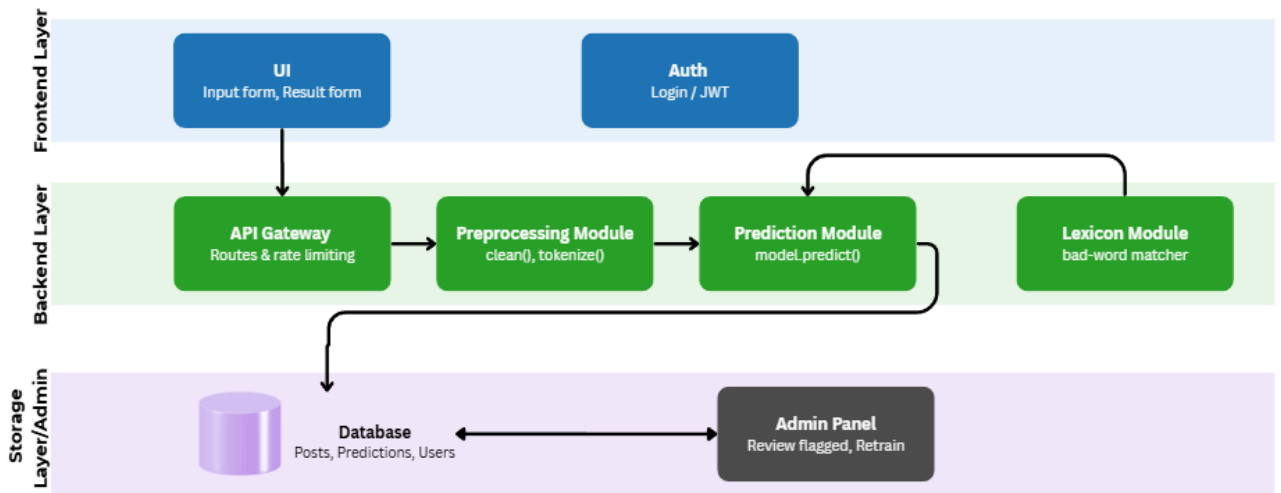


Figure 2: Modular Diagram

- UI (Frontend) sends a post to the API Gateway.
- The API Gateway routes the post to the Preprocessing Module.
- Preprocessing cleans and tokenizes the text.
- The cleaned text moves to the Prediction Module.
- Prediction calculates a toxicity score, using the Lexicon Module for bad-word matching.
- The result (post, prediction, and flags) is saved to the Database.
- The Admin Panel accesses the Database to review flagged items and initiate model retraining.

4.3 Detailed Design

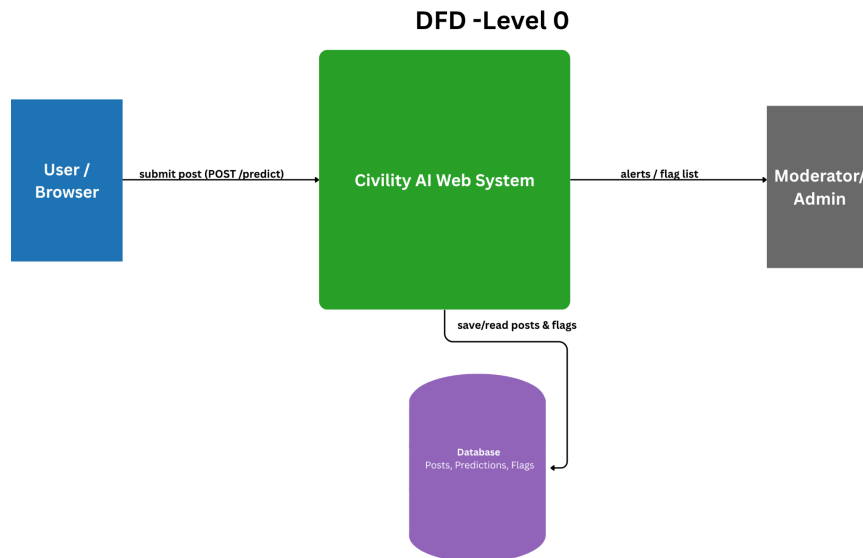


Figure 3: DFD Level 0

- This diagram shows the entire system as a single process and how it interacts with the outside world.
- User → System: The User/Browser submits text posts to the Civility AI Web System to get a prediction.
- System ↔ Database: The system saves and reads all posts, predictions, and flags from the central Database.
- System → Moderator: The system sends alerts and the flag list to the Moderator/Admin for human review.

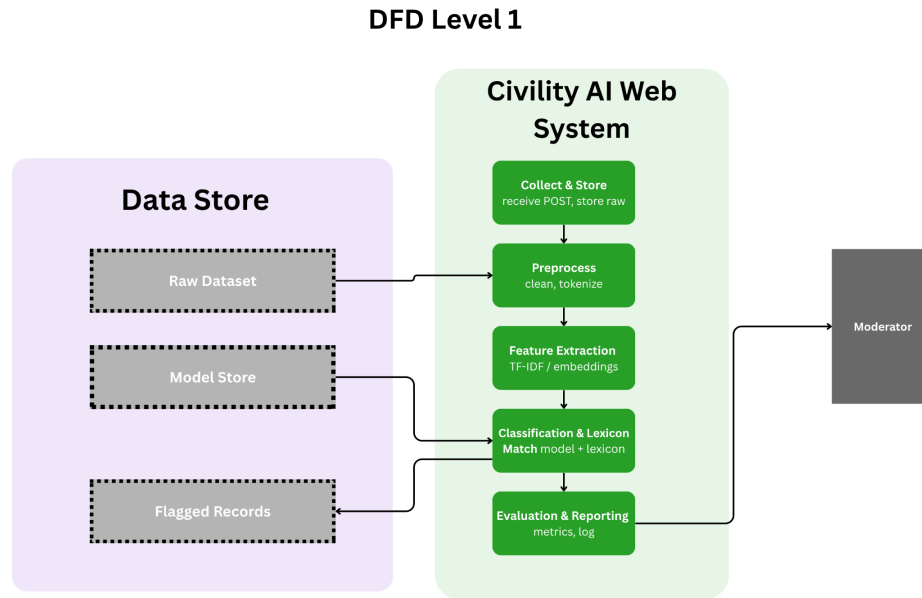


Figure 4: DFD Level 1

- ❖ This diagram breaks the system into five main processes (P1 to P5) and shows the data flow between them and the data stores.
- ❖ P1 & P2 (Receive & Clean): Collect & Store (P1) receives the raw post, which is then sent to Preprocess (P2) to be cleaned and tokenized.
- ❖ P3 & P4 (Analyze): Feature Extraction (P3) prepares the data, which then goes to Classification & Lexicon Match (P4) to get a prediction score. P4 reads from the Model Store.
- ❖ P5 (Report): Evaluation & Reporting (P5) processes the results, logs flagged content to the Flagged Records store, and sends final alerts/reports to the Moderator.

Flowchart

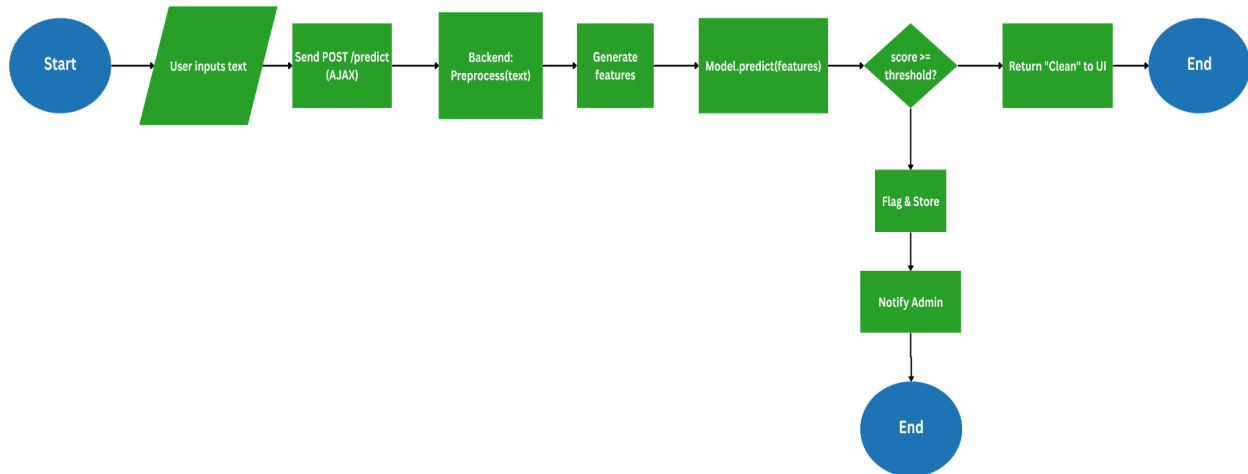


Figure 5: Flowchart

This chart shows the minute-by-minute process flow for a single post submission.

1. **Process Flow:** The process starts with the User Input, which is sent to the Backend.
2. **Analysis:** The backend Preprocesses the text and the Model generates a score.
3. **Decision:** The system checks if the Score $\geq$ Threshold.
4. If Yes: It proceeds to Flag & Store the content and Notify the Admin.
5. If No: It immediately returns "Clean" to the UI, meaning the post is safe.

State Transition Diagram

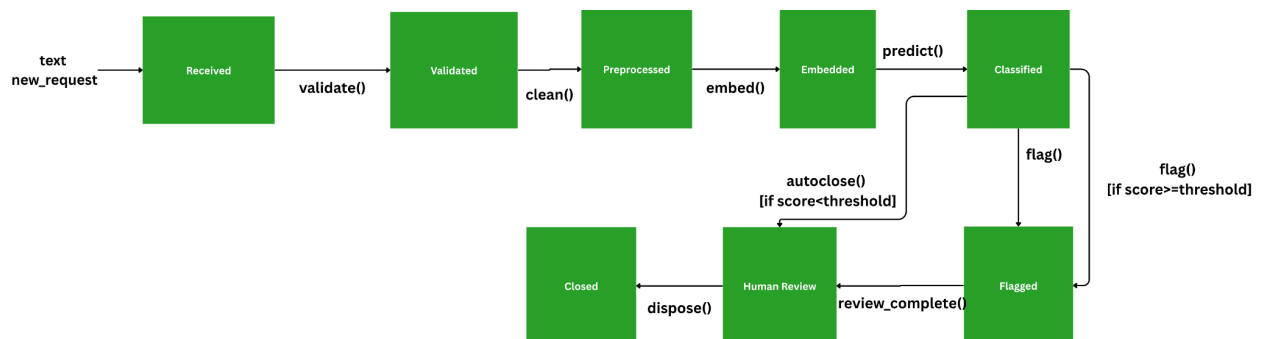


Figure 6: State Transition Diagram

- This diagram tracks the entire lifecycle of a single text instance as it moves through the system's pipeline.
- Initial Flow: A new request moves through preparation states: Received → Validated → Preprocessed → Embedded.
- Classification: It enters the Classified state after the prediction is made.

Divergence:

- Safe Path: If the score is low (score < threshold), it uses the autoclose() transition straight to the Closed state.
- Flagged Path: If the score is high (score $\geq$ threshold), it uses the flag() transition to Flagged.
- Review: From Flagged, it enters Human Review and finally transitions to Closed once disposed of.

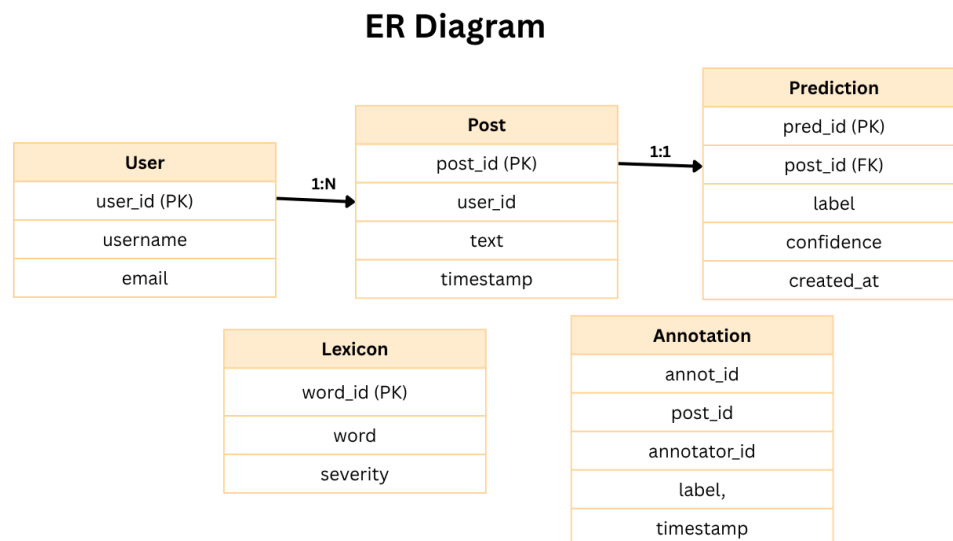


Figure 7: ER Diagram

This diagram maps out the relationships between the key data tables in your system's database.

- ➔ **User → Post (1:N)**: One user can submit many posts.
- ➔ **Post → Prediction (1:1)**: Every post has exactly one prediction score and confidence label associated with it.

- **Annotation:** This table stores the human moderation actions (labels) on a specific post, crucial for model retraining.
- **Lexicon:** A standalone lookup table for known bad words and their severity scores, used during the prediction phase.

4.4 Project Scheduling & Tracking using Time line / Gantt Chart

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9
Topic Discussion & Scope Finalization									
Literature Review (Full Draft)									
Synopsis Submission (Formal Hand-in)									
Requirements Brainstorming (DFD/Flowchart)									
Review 1									
Adding the Backend Logic									
Building of UI/Frontend									
Testing									
Review 2									

Above is the Gantt Chart which conveys the schedule which is planned for the Semester

Chapter 5: Results

Civility AI

Enter text

Check Civility

Refine and Download

Above is the landing page of the sight which can be used to determine if text is foul or not

Chapter 6: Plan of Action for next Semester

6.1. Work done till date

The system for **foul language detection** has been successfully developed and tested using both **individual text inputs** and **bulk text files**. The implementation integrates **Natural Language Processing (NLP)** and **machine learning techniques** to identify and classify offensive or inappropriate language.

Text preprocessing techniques such as **tokenization**, **lemmatization**, and **stopword removal** were applied to clean and normalize the input data. The processed text was then analyzed using the **Perspective API**, which provides a toxicity score and categorization of offensive content based on pre-trained language models.

A simple and interactive **web interface**, built using **HTML, CSS, and Flask (Python)**, allows users to input text manually or upload text files for automated foul language detection. The system returns the analysis results in a user-friendly format, highlighting the detected level of toxicity.

Overall, the current phase demonstrates a fully functional **English-language foul language detection system** that effectively combines NLP preprocessing with cloud-based API intelligence.

6.2. Plan of action for project II

The upcoming phase of the project will focus on **enhancing the system's versatility, scalability, and language coverage**. The following objectives are planned:

1. **Multilingual Support** – Extend foul language detection to include **regional and international languages** such as Hindi, Tamil, and Spanish by integrating translation pipelines or multilingual NLP models.
2. **Expanded Input Formats** – Enable the system to process **social media posts, chat messages, and user comments**, simulating real-world text data scenarios.
3. **Audio Input Handling** – Incorporate **speech-to-text modules** (e.g., Google Speech Recognition API) to detect foul language in **audio recordings or voice inputs**.
4. **Improved Visualization & Reporting** – Enhance the web dashboard to display **detailed analytics**, such as frequency of offensive terms and sentiment trends.
5. **Optimization & Evaluation** – Conduct **performance benchmarking and accuracy testing** using multilingual datasets to evaluate the system's robustness and reliability.

This next phase will transition the current prototype into a **comprehensive, multilingual foul language detection platform** capable of processing diverse text and audio data sources.

Chapter 7 : Conclusions

The project **Civility AI: Foul Language Detection System** presents an effective and scalable approach to identifying and filtering offensive language in online communication. By integrating **Natural Language Processing (NLP)** and **Machine Learning (ML)** techniques, the system successfully automates the detection of abusive and inappropriate text across multiple languages and domains.

The project addresses major challenges such as context ambiguity, multilingual code-mixing, and slang variation through a hybrid modeling approach that combines **TF-IDF**, **GloVe**, and **Word2Vec embeddings**. The use of interpretable machine learning models such as **Logistic Regression**, **Naive Bayes**, and **SVM** ensures transparency and adaptability without sacrificing performance.

Overall, **Civility AI** contributes toward fostering respectful digital interactions by combining efficiency, fairness, and interpretability. With further improvements such as deep-learning integration, transformer-based contextual understanding, and multilingual dataset expansion, the system holds significant potential for future research and deployment in large-scale moderation frameworks.

Chapter 8 : References

I. Books referred

- [1] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA: MIT Press, 1999.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. 3rd ed. London, UK: Pearson Education, 2020.
- [3] C. Albon, *Machine Learning with Python Cookbook*. Sebastopol, CA: O'Reilly Media, 2018.

II. Research Papers referred/ Journals/ Articles referred

- [1] T. Davidson, D. Warmley, M. Macy, and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.04009>
- [2] Z. Waseem and D. Hovy, "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," *Proceedings of the NAACL-HLT Conference*, 2016. [Online]. Available: <https://aclanthology.org/N16-2013/>
- [3] A. M. Founta, C. Djouvas, D. Chatzakou, I. Leontiadis, J. Blackburn, G. Stringhini, A. Vakali, M. Sirivianos, and N. Kourtellis, "Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2018. [Online]. Available: <https://ojs.aaai.org/index.php/ICWSM/article/view/15059>
- [4] P. Mishra, H. Yannakoudakis, and E. Shutova, "Abusive Language Detection with Graph Convolutional Networks," *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. [Online]. Available: <https://aclanthology.org/P19-1035/>
- [5] R. Sharma, D. Patel, and A. Kumar, "Offensive Language Detection on Social Media using Machine Learning," *International Journal of Innovative Research in Computer and Communication Engineering (IJIRCCE)*, vol. 8, no. 3, 2020.

[6] S. Banerjee and P. Roy, “Toxic Comment Classification using Deep Learning Techniques,” *IEEE Access*, vol. 9, pp. 103375–103384, 2021.

[7] A. Gupta and K. Thomas, “A Multilingual Approach for Offensive Language Detection in Code-Mixed Data,” in *Lecture Notes in Computer Science (LNCS)*, Springer, 2022.

Chapter 9: Appendix

a. List of Figures

Figure Number	Heading	Page no.
1	Block Diagram	21
2	Modular Diagram	22
3	DFD Level 0	23
4	DFD Level 1	24
5	Flowchart	25
6	State Transition Diagram	25
7	ER Diagram	26

b. List of tables

Table Number	Heading	Page no.
Table 1	Comparison of Research Papers	8
Table 2	Lacuna of existing system and proposed work	14